

Project 5: Memory Management

Due by **November 22, 11:59PM, 11:59 pm**

Summary

To support multiprogramming, modern OSes present virtual memory to processes so that each process sees a contiguous virtual memory address space which may not be contiguous in physical memory and may not be completely in memory. The OS maintains the mapping between a process' virtual memory and the computer's physical memory through the process' page table. Every page table entry stores the mapping information of a virtual page, indicating whether the page is present in memory and, if yes, the physical frame number it is mapped to. Additional data structures track the temporarily swapped pages from memory to disk.

In this project, we will implement a new kernel function to reveal the “magic” that the kernel does to virtualize memories, using the same virtual machine environment you prepared in the previous projects. This project will help you understand how a real-world OS like Linux performs memory management and master the skills to implement it in kernel space.

Description

In this project; you will implement a kernel module which, for a given process' PID and a virtual memory address in the process' memory space, should walk the process' page table and return the physical memory address that the specified virtual address is mapped to if it is present in physical memory. If the page where this virtual address belongs to is not present in memory but is in swap, it should return the swap identifier of the corresponding page in swap. Otherwise, it should indicate that the page is not available.

1. Module interface and module_init

You **MUST** name your module “**memory_manager**”.

It takes **two** arguments and you **MUST** name your input parameters for the kernel module as follows such that the test script can load the kernel module successfully.

- **pid:** int, pid of a process (a non-negative number)
- **addr:** unsigned long long, a virtual address of the process (a non-negative number)

2. Page Table Walk

Given the virtual address you need to walk the process' page tables to find out if the address is present in physical memory or in the swap.

The Linux kernel (6.10) implements 5 levels of page tables: **PGD, P4D, PUD, PMD, and PTE** (see Figure 1).

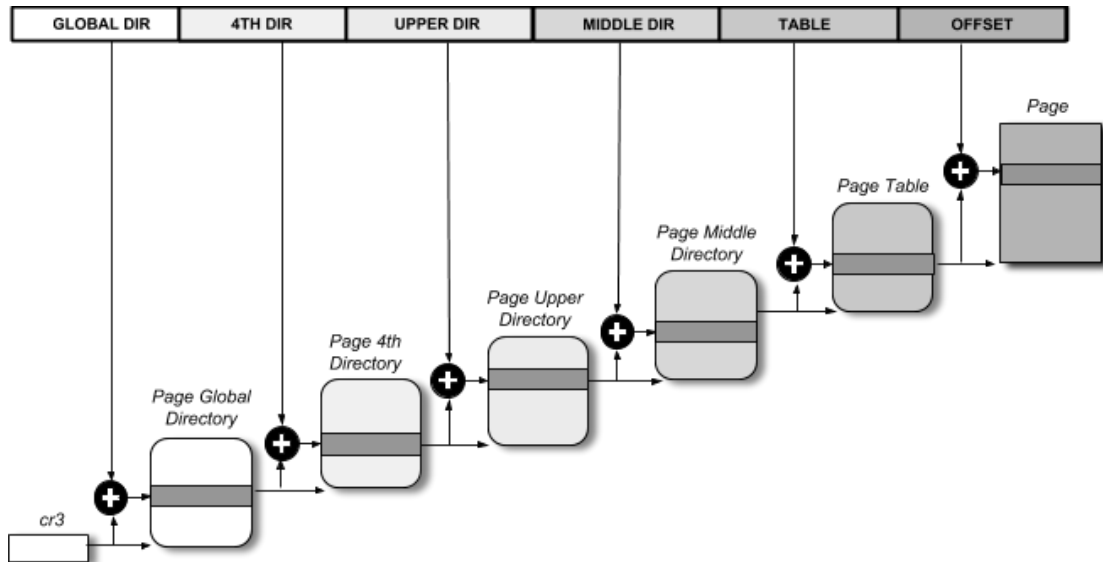


Figure 1

The `mm_struct` of a process' `task_struct` contains the memory management information of the process:

```
#include <linux/mm_types.h>
struct mm_struct* mm = task->mm // where task is a task_struct pointer
```

With the `mm_struct`, we can walk the different levels of page tables using the following functions:

```
#include <linux/mm.h>

pgd_offset(mm, address) /* Return pointer to the PGD. mm is the mm_struct
of the process, address is the virtual address in the virtual memory
space*/

p4d_offset(pgd_t *pgd, unsigned long address) /* Return pointer to the
P4D. pgd is the pointer of PGD, address is the virtual address in the
virtual memory space.*/
```

```

pud_offset(p4d_t *p4d, unsigned long address) /* Return pointer to the
PUD. p4d is the pointer of P4D, address is the virtual address in the
virtual memory space.*/

pmd_offset(pud_t *pud, unsigned long address) /* Return pointer to the
PMD. pud is the pointer of PUD, address is the virtual address in the
virtual memory space.*/

pte = pte_offset_kernel(pmd, address);
/* Return pointer to the PTE. pmd is the pointer of PMD, address is the
virtual address in the virtual memory space*/

```

Below is an example of the page table walk:

```

pgd_t *pgd;
p4d_t *p4d;
pmd_t *pmd;
pud_t *pud;
pte_t *pte;

pgd = pgd_offset(mm, address);
// get pgd from mm and the page address
if (pgd_none(*pgd) || unlikely(pgd_bad(*pgd))) {
    // check if pgd is bad or does not exist
    return;
}
p4d = p4d_offset(pgd, address);
//get p4d from from pgd and the page address
if (p4d_none(*p4d) || unlikely(p4d_bad(*p4d))) {
    // check if p4d is bad or does not exist
    return;
}
pud = pud_offset(p4d, address);
// get pud from from p4d and the page address
if (pud_none(*pud) || unlikely(pud_bad(*pud))) {
    // check if pud is bad or does not exist
    return;
}
pmd = pmd_offset(pud, address);
// get pmd from from pud and the page address
if (pmd_none(*pmd) || unlikely(pmd_bad(*pmd))) {
    // check if pmd is bad or does not exist
    return;
}
pte = pte_offset_kernel(pmd, address);
// get pte from pmd and the page address

```

```
if (!pte) {
    // check if pte does not exist
    return;
}
```

3. Address Translation

After the page table walk, we obtain the PTE for the page that the virtual address belongs to. The next task is to check if this page is present in memory, in swap, or invalid.

a. Page is present in memory

To check if the page is present in memory, we can use the function `pte_present()`:

```
int pte_present(pte_t pte); /* Return 1 if a page table entry is in
the main memory, pte is a pointer to the page table entry.*/
```

If the page is present in memory, the PTE stores the physical number of the corresponding page, named Page Frame Number (PFN), and we can use the function `pte_pfn()` to retrieve the PFN.

```
unsigned long pte_pfn(pte_t pte);
```

With the virtual address and the PFN of its page, we can obtain its physical address by performing bit operations. Hints: 1) We can left-shift the PFN by `PAGE_SHIFT` to get the higher bits (the page number portion) of the physical address; 2) We can use an AND `&` operation to get the lower bits (the offset portion) of the physical address.

Your kernel module should print a message in the following format if the page is present in memory:

```
[CSE330-Memory-Manager] PID [pid]: virtual address [v-addr]  physical address
[p-addr] swap identifier [NA]
```

Example:

```
[CSE330-Memory-Manager] PID [1408]: virtual address [7cd1b0a38b8c]  physical
address [a7cc6b8c] swap identifier [NA]
```

Note: You **MUST** strictly follow the above template to log messages. PID is in **decimal** and virtual/physical addresses are in **hexadecimal**.

b. Page is present in swap

If the page where the virtual address belongs is not in memory but in swap, the function `pte_present()` returns 0, and we can use the `pte_to_swp_entry()` function to convert the PTE to a swap entry and find out the swap identifier. A swap identifier is a unique value used to track and reference the pages that have been moved from memory to swap on disk.

```
swp_entry_t entry = pte_to_swp_entry(pte);
swap_id = entry.val; // val stores the swap identifier
```

Your kernel module should print a message in the following format if the page is present in swap:

[CSE330-Memory-Manager] PID [pid]: virtual address [v-addr] physical address [NA] swap identifier [swap-id]

Example:

[CSE330-Memory-Manager] PID [1418]: virtual address [7d73ef311e83] physical address [NA] swap identifier [c82b]

Note: You **MUST** strictly follow the above template to log messages. PID is in **decimal**, and virtual address and swap identifier are in **hexadecimal**.

c. Invalid page address

If the virtual address's corresponding PTE is NULL, it is not valid, and your kernel module should print a message in the following format:

[CSE330-Memory-Manager] PID [pid]: virtual address [v-addr] physical address [NA] swap identifier [NA]

Example:

[CSE330-Memory-Manager] PID [1427]: virtual address [20d86b11c7ac86c7] physical address [NA] swap identifier [NA]

Note: You **MUST** strictly follow the above template to log messages.

Testing

- In order to test your project we have provided you with the grading scripts. They are available on a GitHub repository: <https://github.com/visa-lab/CSE330-OS.git>
In order to clone the Github repository follow the below steps:
 - `git clone https://github.com/visa-lab/CSE330-OS.git`
 - `cd CSE330-OS/`
 - `git checkout project-5`
- You **MUST** enable swap on your VM. Check if swap is enabled using “`free -h`”; if not you can follow below steps on how to enable swap using a swapfile
 - For example to have 1GB swap, create a 1 GiB file (`/mnt/1GiB.swap`) to use as swap:
`sudo dd if=/dev/zero of=/mnt/1GiB.swap bs=1024 count=1048576`
 - Set the swap file permissions to 600 to prevent other users from being able to read potentially sensitive information from the swap file `sudo chmod 600 /mnt/1GiB.swap`
 - Format the file as swap: `sudo mkswap /mnt/1GiB.swap`
 - Enable use of swap: `sudo swapon /mnt/1GiB.swap`
 - Swap is now available and can also be verified with: `sudo cat /proc/swaps`
 - To enable swap file at bootup add the swap file details to `/etc/fstab` so it will be available at bootup:
`echo '/mnt/1GiB.swap swap swap defaults 0 0' | sudo tee -a /etc/fstab`
- You **MUST** set your VM to use 4GB or less memory to ensure the test scripts can trigger swapping.
- Use [test_module.py](#) to test your module code.
 - The script is written in Python. If you do not already have it installed, you can install it using apt: `sudo apt install python3`
 - The script reads multiple files from `procfs` and `/var/log/kern.log` to validate your code. Hence the script **MUST** always be run with `sudo`.
 - To test virtual addresses that are present in physical memory, the test script will disable swap and allocate 1GB of memory (to ensure they do not get swapped out during the testing).

- To test virtual addresses that are in swap, the test script will enable swap and should perform a much larger memory allocation to ensure pages are being moved to swap. We suggest you allocate as much as the memory size of your VM.
- Below shows an example of the expected output from Test Cases #1, #2, and #3 with a VM given 4GB of RAM:

```
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$ sudo ./test_module.py ../memory_manager.ko 1 100 0 0
[sudo] password for vboxuser:
[log]: Disable swap
[log]: Waiting for 5 seconds to allow pages to be present and/or to be moved to swap
[log]: Checking 100 random present pages
[log]: - 100/100 correct
[log]: Enabling swap
[memory_manager]: Passed (100.0/100)
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$ sudo ./test_module.py ../memory_manager.ko 4 0 100 0
[log]: Enabling swap
[log]: Waiting for 5 seconds to allow pages to be present and/or to be moved to swap
[log]: Checking 100 random swapped pages
[log]: - 100/100 correct
[memory_manager]: Passed (100.0/100)
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$ sudo ./test_module.py ../memory_manager.ko 1 0 0 100
[log]: Disable swap
[log]: Waiting for 5 seconds to allow pages to be present and/or to be moved to swap
[log]: Checking 100 random invalid pages
[log]: - 100/100 correct
[log]: Enabling swap
[memory_manager]: Passed (100.0/100)
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$
```

- Use [test_zip_contents.sh](#) to validate that your zip file adheres to the directory structure that is expected. Below shows an example of the expected output:

```
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$ ./test_zip_contents.sh /tmp/submit/Project5-1225754101.zip
[log]: Look for directory (source code)
[log]: - file /home/vboxuser/git/GTA-CSE330-Fall2024/Project5/test/unzip_1730670215/source_code found
[log]: Look for Makefile
[log]: - file /home/vboxuser/git/GTA-CSE330-Fall2024/Project5/test/unzip_1730670215/source_code/Makefile found
[log]: Look for source file (memory_manager.c)
[log]: - file /home/vboxuser/git/GTA-CSE330-Fall2024/Project5/test/unzip_1730670215/source_code/memory_manager.c found
[test_zip_contents]: Passed
vboxuser@vboxuser-VirtualBox:~/git/GTA-CSE330-Fall2024/Project5/test$
```

- **Note:** We will check your code in addition to running the test script, so passing the testing script does not guarantee getting all the grade points. But if your submission fails to pass the [test_zip_contents.sh](#) script, you will receive **zero** grade points.
- **Note:** We will use an anti-plagiarism tool and AI content detector to check the integrity of your code.

Submission Requirements & Guidelines

- Submit the following in a zip file to Canvas:
 - Source code: Make a directory named `'source_code'`, and include your code file named `'memory_manager.c'` and your makefile named `'Makefile'`.
- You **MUST** name the zip file following the below naming convention: "Project5-<Your ASU ID>.zip", e.g., Project5-1225754101.zip

- You can use the following command to create your zip file:

```
zip -r Project5-1225754101.zip source_code/
```

Note: Extensions automatically generated by Canvas due to multiple submission attempts will be managed by the grading script.

- Do not submit any other source code
- Do not submit any binary
- If your submissions do not adhere to submission guidelines and naming conventions you will receive zero points.

Grading Rubrics

Test Case	Criteria	Points
Test Case #1 The Kernel module is loaded and unloaded 100 times with an address present in memory passed to the module in each iteration.	Input: <pre>sudo ./test_module.py /path/to/memory_manager.ko 1 100 0 0</pre> Expected Output: Kernel module prints the correct physical address. If the module cannot load or unload, deducting all the points of the following testing addresses.	0.3333 point for each correct physical address ----- Total 33.3333 points
Test Case #2 The Kernel module is loaded and unloaded 100 times with an address present in swap passed to the module in each iteration.	Input: <pre>sudo ./test_module.py /path/to/memory_manager.ko 4 0 100 0</pre> Change 4 to match how much memory your VM has. If you have 2GB replace 4 with 2. If you have 3GB replace 4 with 3. Expected Output: Kernel module should print the correct swap identifiers of the page.	0.3333 point for each correct swap identifier ----- Total 33.3333 points

	If the module cannot load or unload, deducing all the points of the following testing addresses.	
Test Case #3 The Kernel module is loaded and unloaded 100 times with an invalid address passed to the module in each iteration.	Input: <pre>sudo ./test_module.py /path/to/memory_manager.ko 1 0 0 100</pre> Expected Output: Kernel module should print the required message for invalid page addresses. If the module cannot load or unload, deducing all the points of the following testing addresses.	0.3333 points for each correct invalid address detected ----- Total 33.3333 points
Submission Content	<pre>./test_zip_contents.sh /path/to/zip/file.zip</pre> 1. Zip file should contain only raw outputs (screenshot), source code. No binaries. 2. All files adhere to the directory structure specified in the project spec.	-5 pts unwanted files or binaries Failure to adhere to the directory structure will result in zero.

Policies

- Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
- Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.
- **The use of generative AI tools is not allowed** to complete any portion of the assignment.

